

2.3 LSTMs and GRUs

An additive cell path. Copy when forget is open and input is closed. GRU is two gates.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Gates are learned valves. They let a recurrent cell **keep** a bit of memory for many steps instead of overwriting it every time. LSTM and GRU are the two you will meet. They do not remove vanishing gradients; they make a near-1 path **available**.

First time this method appears. LSTMs and GRUs are gated RNNs.

What. An additive **cell** path that can copy a value when the forget gate is open and the input gate is closed. GRU is the two-gate cousin. **Why.** Vanilla RNNs vanish. You need a highway whose gradient is ≈ 1 along the copy. **Architecture.** LSTM: cell c_t , hidden h_t , gates i, f, o . GRU: reset and update, no extra c . **How.** Copy regime: $f \approx 1, i \approx 0 \Rightarrow c_t \approx c_{t-1}$. Lab 2: LSTM vs RNN on a long copy. **Formula.** $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $h_t = o_t \odot \tanh(c_t)$. GRU: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. **Tradeoffs.** + Long-range copy, still sequential. – More parameters than a vanilla RNN; still not parallel like attention. GRU is cheaper, sometimes enough.

1. The problem gates are for

In a vanilla RNN the hidden state is **constantly rewritten** by $\tanh(W_h h_{t-1} + \dots)$. CMU 11-785's stability lecture: unless W_h is essentially a wire (eigenvalues on the unit circle and no saturating tanh), memory is short. You cannot ask SGD to discover a unitary W_h and a linear activation at once.

What you want instead: a **cell** that can copy, plus **input-dependent switches** that say when to write, when to erase, and when to report. That is an LSTM (Hochreiter and Schmidhuber 1997; forget gate from Gers et al. 2000).

2. LSTM: a highway plus three gates

A **cell state** c_t moves down the chain with mostly multiplies and adds, not a stack of tanhs. Three sigmoid gates, each in $(0, 1)$:

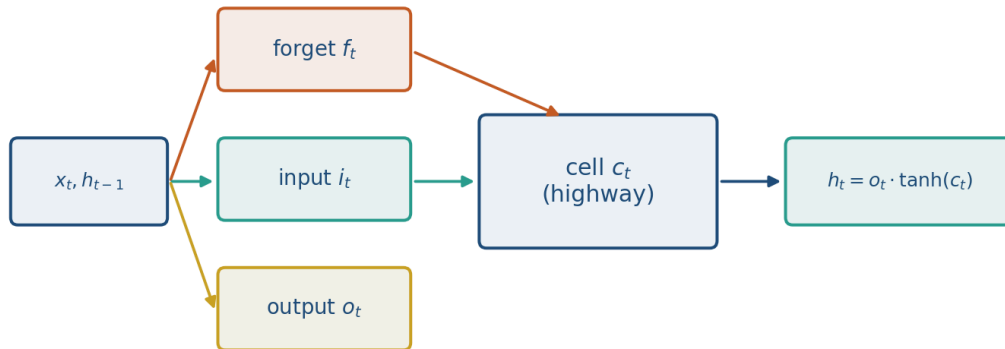
Gate	Role
Forget f_t	How much of c_{t-1} to keep
Input i_t	How much new candidate to write
Output o_t	How much of $\tanh(c_t)$ becomes h_t

Each gate is a sigmoid of a linear mix of x_t and h_{t-1} (plus a bias). You do not need every matrix name on the first pass. You need the cell update:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t).$$

The candidate $\tilde{c}_t$ is a tanh of a linear mix of x_t and h_{t-1} .

LSTM: a cell highway plus learned gates



If $f_t \approx 1$ and $i_t \approx 0$, the cell **copies**. That is the long-range path. Along that coordinate,

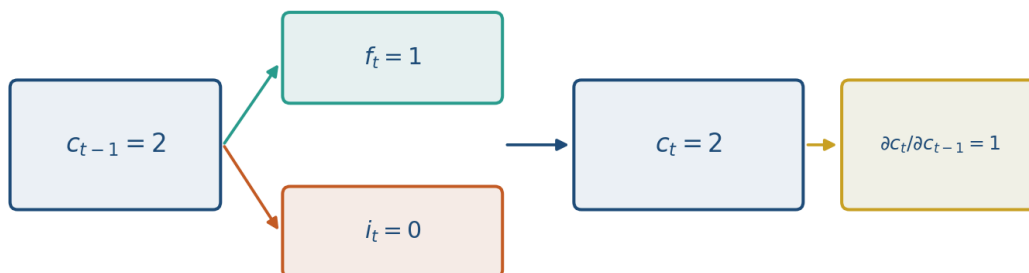
$$\frac{\partial c_t}{\partial c_{t-1}} = f_t,$$

which can stay near 1. That is why the vanishing plot in 2.2 had a gated curve. CS231N's warning, which we keep: **LSTM does not guarantee** that every coordinate copies. It makes a copy path *learnable*. In practice you get on the order of 10^2 steps rather than ≈ 7 , not infinite context.

Peephole connections (cell into the gates) exist in some papers. We do not use them. `nn.LSTM` is the three-gate cell above.

3. The copy regime, in numbers

Copy regime: forget open, input closed. The cell is a wire.



Scalar cell (drop the output gate for a minute). Suppose $c_{t-1} = 2$, $f_t = 0.9$, $i_t = 0.1$, $\tilde{c}_t = 5$:

$$c_t = 0.9 \cdot 2 + 0.1 \cdot 5 = 1.8 + 0.5 = 2.3.$$

Most of the old memory survived; a little new content landed. If instead $f_t = 0$, $c_t = 0.5$ and the 2 is gone.

Copy regime: $f_t = 1$, $i_t = 0 \Rightarrow c_t = c_{t-1}$. Ten such steps: $c_{t+10} = c_t$. Gradient along that path is $1^{10} = 1$.

That is Lab 2's hope: the first bit sits in c and the forget gate learns to leave it there until the last step.

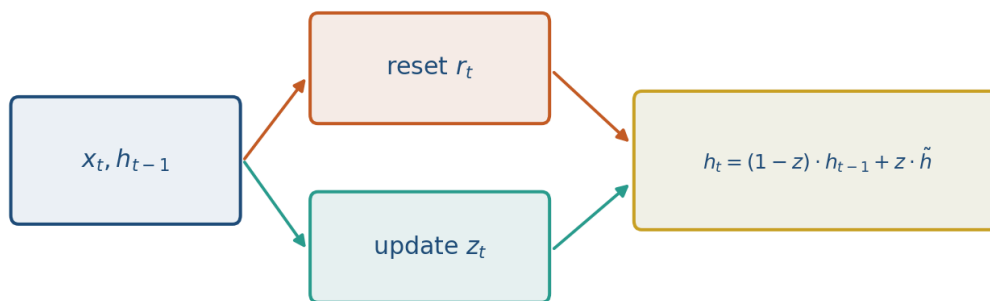
4. GRU: two gates, no extra cell

Cho et al. (2014). Reset r_t and update z_t . The hidden state *is* the memory.

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

Read z_t as "how much to replace." If $z_t \approx 0$, you copy h_{t-1} . Reset r_t controls how much past goes into the candidate $\tilde{h}_t$.

GRU: two gates, no separate cell state



Fewer parameters, often similar quality on medium sequences. Use LSTM when you want an explicit cell; use GRU when you want a smaller default. Both are still **sequential**. You still cannot fill a GPU the way a transformer can. Long context is better than vanilla, worse than attention.

For this course, gates are the reason we can talk about "memory" before Week 3, and a baseline in speech/time-series projects.

5. Bidirectional and stacked

Bidirectional LSTM: run one net forward and one backward, concatenate. Fine for classification when the whole sentence is allowed. **Illegal** for next-token generation (you would peek at the future).

Stacked LSTM: h_t of layer ℓ is the input to layer $\ell + 1$. Deeper mixing, still a loop. Do not stack four layers in Lab 2.

6. Teaching this note

~**16 minutes**. Table of three LSTM gates, then the copy regime $f \approx 1, i \approx 0$, then the numeric cell update. GRU as “two knobs, hidden state is the memory.” One sentence on bidirectional-is-illegal-for-LMs. Play StatQuest LSTM **0:00–12:00** (forget/input intuition). Assign the GRU video as a 10-minute clip after class, or play **0:00–8:00** if the room is still with you.

7. Worked example

Already in §3 for the LSTM cell. GRU: $h_{t-1} = 4, z_t = 0.25, \tilde{h}_t = 0$ gives

$$h_t = (1 - 0.25) \cdot 4 + 0.25 \cdot 0 = 3.$$

You kept 75% of the past.

Vector GRU with $h_{t-1} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, z_t = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \tilde{h}_t = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \odot \begin{bmatrix} 2 \\ 5 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}.$$

Coordinate 1 replaced ($z = 1$); coordinate 2 copied ($z = 0$).

8. Where students get stuck

- Mixing up forget vs. input (zeroing f when they meant “write nothing new”).
 - Claiming LSTMs “solved vanishing gradients” in every coordinate, always.
 - Using a bidirectional LSTM for a generator that must not see $t + 1$.
 - Drawing a new forget-gate matrix at each time step (gates share weights too).
-

9. Video

Watch [StatQuest: Long Short-Term Memory \(LSTM\), Clearly Explained](#).

Pause when the forget gate is a number in $(0, 1)$ multiplying the cell, and when the cell is drawn as a highway. Then [StatQuest: Gated Recurrent Units \(GRU\), Clearly Explained](#) — play the update-gate mix; skip if time is gone.

The matching university lecture is CMU 11-785 Lecture 14 (stability and memory, then the LSTM as a constant-error carousel). Stanford CS224N’s LSTM hour (older “fancy RNN” lecture) is the copy-regime slogan we use on the board. We do not copy either deck.

10. Practice

1. If the forget gate is stuck at 0, what happens to c_t ?
2. Why is a bidirectional LSTM the wrong cell for a language model that must emit token $t + 1$?
3. Name one reason you might still pick a GRU over a transformer in a project.
4. Compute c_t for $c_{t-1} = 1$, $f_t = 0.5$, $i_t = 0.5$, $\tilde{c}_t = -1$. Then compute h_t if $o_t = 1$ and $h_t = o_t \tanh(c_t)$.
5. GRU with $h_{t-1} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}$, $z_t = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\tilde{h}_t = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$. Compute h_t . Which coordinate copied, and which replaced?
6. In one line: why can $\partial c_t / \partial c_{t-1}$ stay near 1 when a vanilla $\partial h_t / \partial h_{t-1}$ usually cannot?
7. Does an LSTM *guarantee* that the first token of a 500-word review still trains? What would you measure in Lab 2 to check?